

대학생 자작자동차대회

기술아이디어 보고서

■ Summary

기존 차량의 모터 및 인버터 세팅은 모터 데이터시트, 인버터 매뉴얼 및 이전 팀원의 경험을 기반으로 수행되었다. 그러나 PM100DX의 Break_Speed, Iq Limit, Torque Limit, DC Bus Current Limit(DCL)은 서로 독립적인 값이 아니라 배터리 전압, 전압 강하, 모터 회전수 및 토크 요구량에 따라 동시에 변하는 값이므로, 개별 수작업 계산만으로는 세팅값 누락, 입력 편차 및 과전류 위험을 사전에 확인하기 어렵다.

25 시즌 차량 개발 과정에서는 파라미터 설정 오류로 컨트롤러 내부에 과전류가 흐르며 모터 컨트롤러 내부 소자가 손상되는 사례가 있었다. 본 설계는 이 문제를 해결하기 위해 단순 모터 성능 계산기가 아니라, 실차 적용 전 PM100DX 초기 세팅 후보의 전류 여유도와 모터 맵 경향을 검토하는 오프라인 세팅 검증 도구를 개발하는 것을 목표로 하였다.

본 도구는 PMSM 전력 관계식을 기반으로 DC Bus Current를 1차 추정하고, 2023년 EMRAX 208MV 원본 CAN 로그를 이용해 실제 차량의 계측 오프셋과 제어 특성을 반영한 회귀 보정식을 도출하였다. 이후 EMRAX 228MV 데이터시트 상수, 배터리 전압 강하, DCL, 목표 토크 및 차량 제원을 입력하여 Break_Speed, Iq 초기값, DCL 사용률 및 RPM 별 모터 맵을 산출하도록 구성하였다. 이를 통해 팀 내 경험 의존적 세팅을 줄이고, 반복 가능한 초기 세팅 검증 절차를 마련하고자 한다.

■ Conceptual Design

2.1 개발 목표 정의

개발 목표는 인버터의 초기 세팅의 경험 의존도를 낮추고, 실차 로그 기반 보정과 이론식 기반 계산을 결합하여 반복 가능한 세팅 검증 절차를 만드는 것이다. 본 설계의 목표는 다음과 같다.

- 별도 설치 없이 팀원이 쉽게 접근할 수 있는 HTML 기반 실행 환경 구성
- 2023년과 2025년 EMRAX 208MV 원본 CAN 로그를 이용한 DC Bus Current 예측 모델 보정
- 배터리 전압, 전압 강하율, DCL, Torque Limit 및 Iq Limit을 동시에 고려한 Break_Speed 후보 계산
- 추천 파라미터 계산과 수동 모터 맵 검토 기능의 분리
- DCL 사용률과 전류 여유도를 표시하여 세팅 후보의 위험도를 사전 판단

- 추후 228MV 실차 로그 확보 시 회귀 계수와 상수만 교체해 갱신 가능한 구조 구성

2.2 대상 시스템 및 데이터 흐름

대상 시스템은 A-FA 의 전기 파워트레인 구성인 배터리 팩, BMS, Cascadia Motion 계열 인버터, EMRAX 계열 PMAC 모터이다. CAN 데이터는 BMS 와 인버터에서 취득하며, 모터 자체에서 직접 CAN 데이터를 수집하는 구조는 아니다. BMS 에서는 배터리 전압 및 직류 제한(이하 DCL)과 관련된 정보를 확인하고, 인버터에서는 DC Bus Current, q 축 전류, RPM, DC Bus Voltage, Torque Feedback 등을 확인한다.

과거 주행 로그는 EMRAX 208MV 기반이며, 현재 차량 적용 대상은 EMRAX 228MV 이다. 따라서 208MV 데이터는 회귀 보정식 검증과 제어 경향 확인에 활용하고, 228MV 계산은 데이터시트 기반 K_e , K_t , 최대 토크 및 최대 RPM 을 적용한 초기 추정값으로 제한한다.

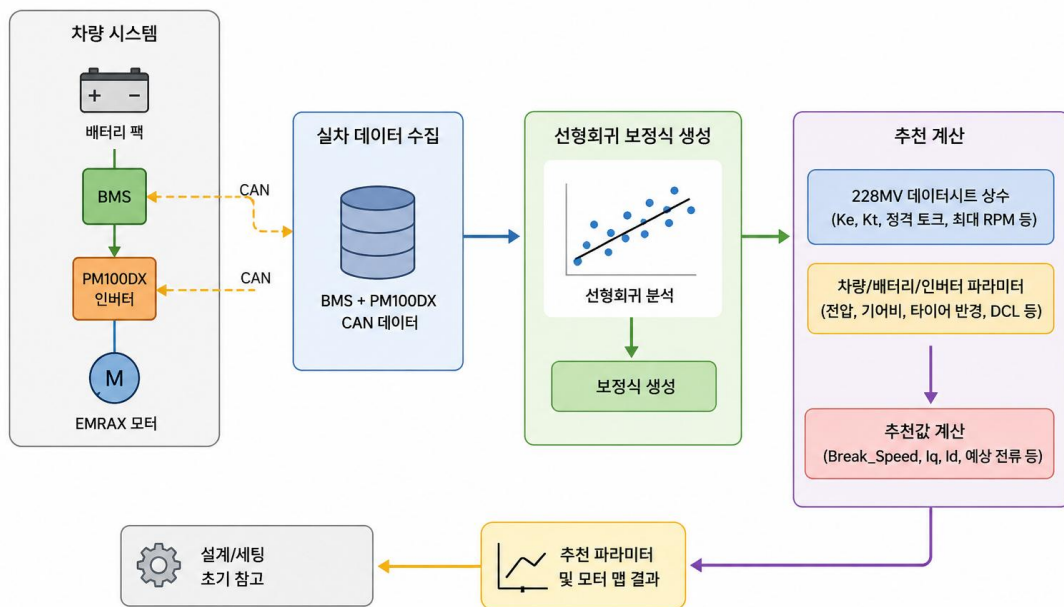


그림 1. 프로그램 개념 및 데이터 흐름

2.3 개발 환경 선정 근거

사용자가 사용하기 편하게 제작하는 프로그램의 목표에 따라 팀원이 별도의 개발 환경을 구축하지 않아도 사용할 수 있어야 하며, 계산식과 상수를 다음 시즌에 수정할 수 있어야 한다. 따라서 C/C++, Python, MATLAB, Excel, HTML/JavaScript 를 비교하였다.

구분	C/C++	Python	MATLAB	Excel	HTML/JavaScript
접근성	컴파일 또는 실행 파일 필요	Python 설치 필요	라이선스 필요	Excel 설치 필요	브라우저로 실행 가능
설치 부담	높음	중간	높음	중간	낮음
UI 구현	직접 구현 필요	GUI 라이브러리 필요	가능하나 무거움	셀 기반 제한	입력창·탭·그래프 구현 용이
계산식 수정	재컴파일 필요	쉬움	쉬움	셀 참조 오류 가능	JS 상수·함수만 수정
배포/인수인계	환경 차이 영향	패키지 버전 영향	라이선스 영향 및 접근 난이도 높음	파일 수식 손상 가능	파일 전달만으로 실행
최종 판단	사용성 낮음	분석용 적합	접근성 제약	프로그램화 한계	최종 구현 방식으로 적합

비교 결과, 파이썬은 주행 데이터 전처리와 선형회귀 분석에 사용하고, 최종 사용자 프로그램은 별도 설치 없이 실행 가능한 HTML/CSS/JavaScript 기반으로 구현하였다. HTML은 화면 구성과 입력 필드를 담당하고, JavaScript는 모터 상수, 회귀식, 추천 계산식, 모터 맵 계산 및 그래프 출력을 하계끔 설계하여 UI, 연산 로직 등 하나의 코드가 길어지고 복잡해지지 않게 제작하였다.

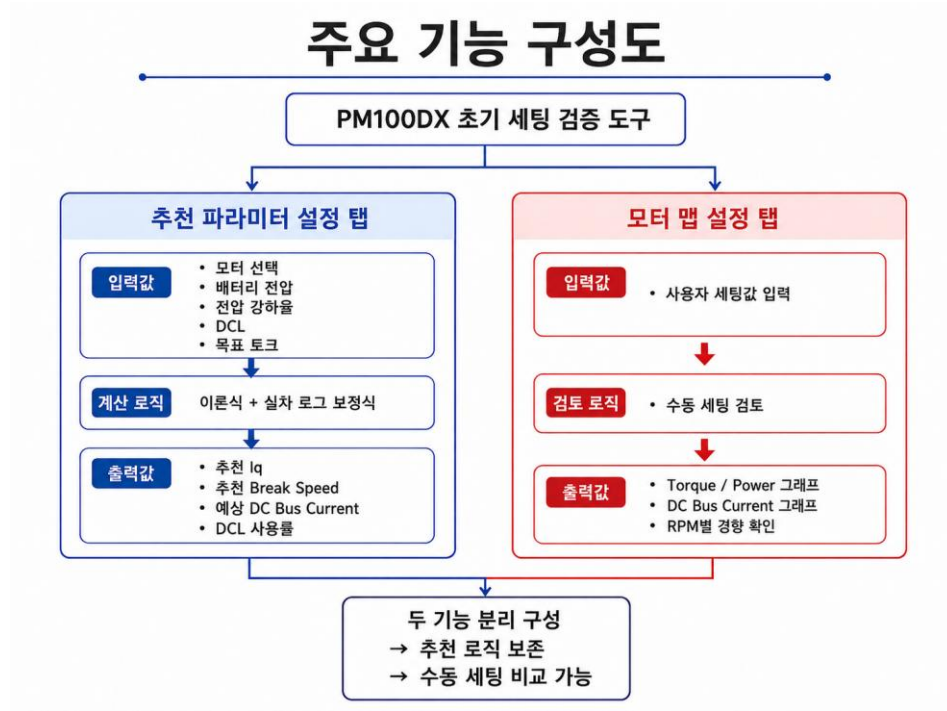
2.4 기존 방식 대비 개선점

기존 방식	본 프로그램 적용 후
데이터시트와 경험 기반의 수작업 계산	PMSM 이론식과 실차 CAN 로그 보정 기반 계산
Break_Speed, Iq, DCL 을 개별 검토	전압, 전류, 토크, RPM 관계를 동시에 검토
과전류 위험 사전 확인이 어려움	예상 DC Bus Current 와 DCL 사용률 확인
세팅 근거가 인수인계 자료에 흩어짐	계산식, 상수, 회귀계수를 코드에 통합 관리
그래프 비교를 별도 Excel/Python 에서 수행	모터 맵과 전류 변화를 즉시 시각화

2.5 주요 기능 구조

프로그램은 추천 파라미터 설정 기능과 설정값 기반 추정 모터 맵 기능을 분리하였다. 추천 탭은 모터 종류와 차량 조건을 입력하면 Break_Speed, Iq, 예상 DC Bus Current, DCL 사용률 기준의 안전 운용 Break_Speed 를 계산한다. 수동 모터 맵 탭은 사용자가 직접 입력한 Break_Speed, Iq Limit, Id Limit, Torque Limit 으로 RPM 별 토크와 DC Bus Current 를 확인하는 기능이다.

두 기능을 분리한 이유는 검증된 추천 로직이 사용자의 실험 입력값에 의해 변형되지 않도록 하기 위함이다. 추천값은 수동 모터 맵 탭으로 복사할 수 있도록 하여, 복사 이후에는 사용자가 독립적으로 값을 수정하도록 구성하였다.



▪ Preliminary Design

3.1 데이터 신뢰도 분류 및 전처리

본 설계에서는 모든 주행 로그를 동일한 신뢰도로 취급하지 않았다. 원본 CAN 메시지 timestamp 가 보존된 2023 로그는 회귀계수 산정 및 정량 검증에 사용하고, 2025 가속 이벤트 로그는 외부 조건에서의 경향 확인에 사용하였다. 반면 timestamp 동기화가 불명확하거나 전류값 고정 및 이상값이 확인된 일부 짐카나 로그는 정량 검증에서 제외하였다.

데이터 종류	신뢰도	사용 목적	비고
2023 EMRAX 208MV 원본 CAN 로그	A	회귀계수 산정 및 정량 검증	메시지별 timestamp 기반 정렬 가능, 60ms 단위 저장
2025 가속 이벤트 로그	B	외부 주행 조건 보조 검증	0.2 s 단위 저장으로 정밀 동기화 제한
일부 짐카나 Excel 로그	C	경향 확인 또는 제외	전류 고정, timestamp 불일치, 이상값 다수

기본 필터 조건은

$V_{dc} > 250 \text{ V}$

$500 \leq \text{RPM} \leq 3900$

$I_q > 20 \text{ A}$

Torque Feedback > 20 Nm

으로 설정하였다. 이 조건은 저속·저전류 구간에서 상대 오차가 과도하게 확대되는 문제와 비정상 출력 저하 구간을 줄이기 위한 것이다.

```
# 주행 데이터 로그 확인
with open(DATA_FILE, "rb") as f:
    records = msgpack.unpack(f, raw=False, strict_map_key=False)

# 신호 추출
current = get_channel(records, "CAN", "CAN_INV_CURRENT", "dc_bus_current", "actual_current")
iq = get_channel(records, "CAN", "CAN_INV_FLUX", "Iq_feedback", "iq")
rpm = get_channel(records, "CAN", "CAN_INV_MOTOR_POS", "motor_speed", "rpm")
vdc = get_channel(records, "CAN", "CAN_INV_VOLTAGE", "dc_bus_voltage", "vdc")
torque = get_channel(records, "CAN", "CAN_INV_TORQUE", "torque_feedback", "torque_feedback")

# 신호 묶어보기 (60 ms)
dt = current

for signal in [iq, rpm, vdc, torque]:
    dt = pd.merge_asof(
        dt,
        signal,
        on="timestamp",
        direction="nearest",
        tolerance=60
    )

dt = dt.dropna()

# 조건식 잡아보기
dt = dt[
    (dt["vdc"] > 250) &
    (dt["rpm"] >= 500) &
    (dt["rpm"] <= 3900) &
    (dt["iq"] > 20) &
    (dt["torque_feedback"] > 20)
].copy()
```

그림 2. CAN 로그 추출 및 전처리 코드 예시

3.2 DC Bus Current 이론식 및 회귀 보정식

DC Bus Current 예측은 주행 데이터만을 이용한 블랙박스형 구현 모델이 아니라 PMSM 전력 관계식 기반의 1 차 모델을 기본으로 하였다. EMRAX 데이터시트에서 역기전력이 회전수에 비례한다고 보고, 선간 전압을 $K_e \times \text{RPM}$ 으로 두었다. I_q 전류를 RMS 전류로 변환하면 3 상 전력은 다음과 같이 근사할 수 있다.

$$I_{DCBUS} \approx \sqrt{\frac{3}{2}} \times \frac{K_e \times I_q \times \text{RPM}}{V_{eff}}$$
$$V_{eff} = V_{DC} \left(1 - \frac{\text{전압강하율}}{100}\right)$$

해당 이론식은 정상상태, $I_d \approx 0$, L_d 와 L_q 차이에 따른 릴럭턴스 토크 영향 무시, 손실항 1 차 제외를 가정한 근사식이다. 릴럭턴스의 영향을 무시한 까닭은 208MV 의 d 축 전류의 직접적인 제어 로직 확인 불가 및 심한 리플, 릴럭턴스 값이 다이노 결과에 따라 $2.4\mu H$ 정도라 영향이 미비하기에 오차가 발생할 것 같아 제외하였다.

실차 적용 시 센서 오프셋, 인버터 내부 소비 전류, 구리손실 및 측정 지연이 존재할 수 있으므로 다음과 같은 선형회귀 보정식을 병행하였다.

$$I_{DC} = b + a(\frac{RPM \times I_q}{V_{eff}})$$

현재 프로그램에 반영된 208MV 보정 계수는 $b = 2.1782$, $a = 0.034141$ 이다. 절편 b 는 저전류 영역의 계측 오프셋과 단순화된 이론식에서 설명하지 못하는 기본 전류 성분을 보정하는 항이며, 기울기 a 는 RPM·Iq·유효전압 조합에 대한 배터리 측 전류 변화 민감도를 의미한다.

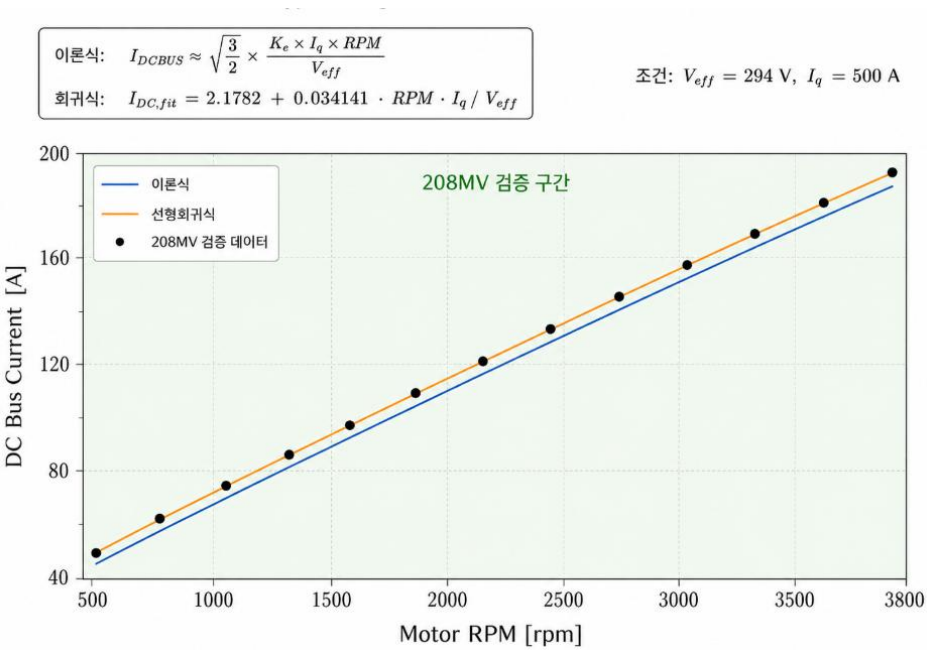


그림 3. 동일 Veff 및 Iq 조건에서 이론식과 회귀식 비교

3.3 회귀식 검증 결과

2023 년 208MV 주행 로그는 원본 CAN timestamp 를 기반으로 DC Bus Current, Iq Feedback, Motor RPM, DC Bus Voltage, Torque Feedback 을 ±60 ms 범위에서 동기화하였다. 필터링 후 시간순으로 80%를 학습, 20%를 검증 데이터로 분리하였다.

모델	MAE	RMSE	R²	MAPE

PMSM 이론식	2.04 A	2.84 A	0.9769	4.42%
선형회귀 보정식	1.76 A	2.24 A	0.9857	4.39%

이론식만으로도 평균 절대 백분율 오차가 4.42%로 나타나 설정 운전 범위 내 전류 변화 경향을 충분히 설명할 수 있었다. 선형회귀 보정식은 MAPE 개선 폭은 크지 않았으나, MAE와 RMSE를 줄여 실차 계측 오프셋과 제어 특성을 반영하는 보정 모델로 활용할 수 있음을 확인하였다.

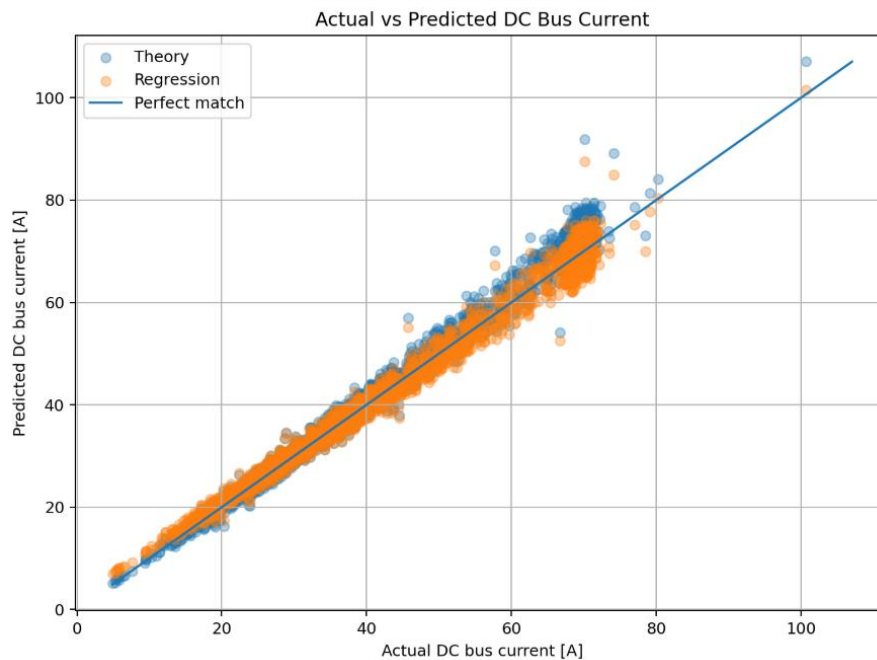


그림 4. 2023 년 208MV 주행 로그 기반 DC Bus Current 예측 결과

2025 년 가속 이벤트 로그는 원본 CAN 메시지 단위 timestamp 가 아닌 0.2 s 단위 저장 데이터이므로 2023 원본 CAN 로그와 동일한 수준의 정량 검증 자료로 사용하기에는 한계가 있다. 따라서 회귀계수 산정에는 사용하지 않고, 별도 주행 조건에서 전류 변화 경향이 유지되는지 확인하는 보조 검증 자료로 활용하였다.

구분	MAE	RMSE	R ²	MAPE
2025 가속 이벤트 보조 검증	4.21 A	4.92 A	0.9273	8.87%

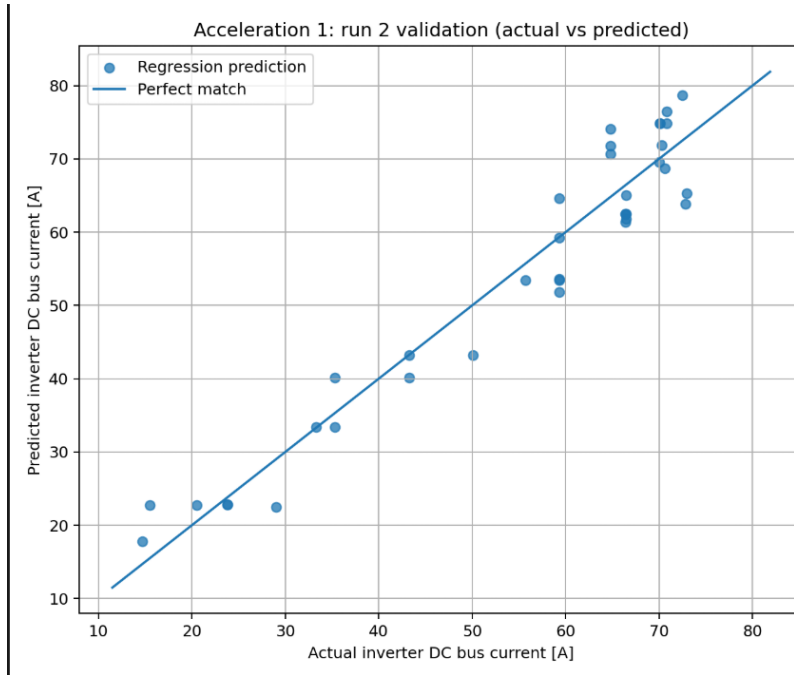


그림 5. 2025 년 가속 이벤트 로그 보조 검증 결과

3.4 DCL 사용률 기반 안전 여유도 판단

이 프로그램에서는 DC Bus Current 가 사용자가 입력한 DCL 대비 어느 정도의 비율을 차지하는지 표시하도록 설계했다. 이는 단순 계산값 표시가 아니라 세팅 후보의 위험도를 사전에 판단하기 위한 자료이다.

$$\text{DCL 사용률} = \text{예상 DC Bus Current} / \text{입력 DCL} \times 100$$

DCL 사용률	판정	의미
0~85%	안전 여유 있음	초기 세팅 후보로 검토 가능
85~100%	검토 필요	전압 강하, 온도, 실차 로그 확인 필요
100% 초과	위험	DCL 초과 가능성이 있어 세팅 조정 필요

현재 프로그램에는 기본 목표 DCL 사용률 90%가 적용되어 있으며, 이를 기준으로 안전 운용 Break_Speed 를 별도로 계산한다. 원래 Break_Speed 추천값은 기존 계산 로직을 유지하고, 안전 운용 Break_Speed 는 DCL 여유도를 확인하기 위한 보조 자료로 제공한다.

3.5 Break_Speed 결정 알고리즘

Break_Speed 는 일정 토크 영역에서 일정 출력 영역으로 전환되는 기준이다. 본 프로그램은 전력 제한과 전류 제한을 동시에 계산하고, 더 낮은 RPM 을 보수적인 Break_Speed 후보로 선택한다.

- 입력 전압과 전압 강하율로 V_{eff} 계산

- 목표 토크와 K_T 로 I_q 요구값 계산

V_{eff} 와 DCL 로 허용 가능한 DC 전력 한계 $P_{DCL} = V_{eff} \times I_{DCL}$ 계산

- 전력 제한 기준 $Break\ Speed = \frac{60 \times P_{DCL}}{2\pi T}$ 계산

- 회귀식 기준 DCL 도달 RPM = $\frac{(I_{DCL}-b) \times V_{eff}}{(a \times I_q)}$ 계산

- Max RPM, 전력 제한 RPM, 전류 제한 RPM 중 가장 낮은 값을 Break_Speed 추천값으로 선택

- 목표 DCL 사용률 기준 안전 운용 Break_Speed 별도 계산

그림 6. Break_Speed 결정 알고리즘 흐름도

▪ Detail Design

4.1 프로그램 구성 및 파라미터 데이터베이스

프로그램의 수정 가능 지점은 JavaScript 코드 상단의 SETTINGS 객체에 집중적으로 배치하였다. SETTINGS 에는 EMRAX 208MV 및 228MV 의 K_e , K_t , 최대 토크, 최대 RPM, 기본 전압 및 기본 DCL 이 저장되어 있다. 208MV 주행 로그 회귀 계수와 228MV 적용 변환 비율도 별도 객체로 분리하여 추후 실차 데이터가 확보되면 값만 교체할 수 있도록 구성하였다.

계산 함수는 추천 탭의 Recommend(), 수동 모터 맵 탭의 calculateManualMap() 및 manualPoint()로 분리하였다. 화면 렌더링과 그래프 함수는 계산 함수와 분리되어 있으므로, 이론식을 개정해도 UI 이벤트나 그래프 기능의 수정 범위를 최소화할 수 있다.

```
# 선형 회귀 : 실제 전류 = 절편 + 기울기 × (RPM × Iq / Vdc)
model = LinearRegression()
model.fit(train[["theory_torque"]], train[["actual_current"]])

test[["maybe_current"]] = model.predict(test[["theory_torque"]])

def show_metrics(name, actual, predicted):
    mae = mean_absolute_error(actual, predicted)
    rmse = mean_squared_error(actual, predicted) ** 0.5
    r2 = r2_score(actual, predicted)
    mape = (np.abs((actual - predicted) / actual).mean()) * 100

    print(f"\n[{name}]\n")
    print(f"MAE = {mae:.2f} A")
    print(f"RMSE = {rmse:.2f} A")
    print(f"R² = {r2:.4f}")
    print(f"MAPE = {mape:.2f} %")

print("Regression equation")
print(
    f"DC bus current = {model.intercept_:.4f} "
    f"+ ({model.coef_[0]:.5f}) × (RPM × Iq / Vdc)"
)

show_metrics("Theory equation", test[["actual_current"]], test[["theory_current"]])
show_metrics("Linear regression", test[["actual_current"]], test[["maybe_current"]])
```

그림 7. 파이썬을 통한 회귀식 계산 및 오차 지표 산출 코드 예시

```

const SETTINGS = {
  motors: {
    emrax208: {
      label: "EMRAX 208 MV",
      ke: 0.0300,
      ktTheory: 0.3507,
      ktRoad: 0.3903366247013316,
      peakTorque: 150,
      maxRPM: 7000,
      defaultVdc: 280,
      defaultSag: 0,
      defaultTorque: 140,
      defaultDcl: 80
    },
    emrax228: {
      label: "EMRAX 228 MV",
      ke: 0.0478,
      ktTheory: 0.5325,
      peakTorque: 240,
      maxRPM: 5500,
      defaultVdc: 300,
      defaultSag: 20,
      defaultTorque: 50,
      defaultDcl: 80
    }
  },

  // 2023 EMRAX 208 filtered-drive-log fit:
  // DC Bus Current = b + a * RPM * Iq / Vdc_eff
  roadFit208: {
    currentSlope: 0.03414106768311502,
    currentIntercept: 2.178222174984981,
    currentTransferRatioFor228: 0.9292021677495167
  },
  recommendationValidation: {
    defaultDclUsePct: 90
  },

  // 모터 맵 기본설정
  manualMap: {
    vdc: 294,
    sag: 20,
    breakRPM: 1800,
    iqLimit: 425,
    idLimit: 0,
    torqueLimit: 231,
    maxRPM: 5500
  }
};

```

그림 8. JS 파일에서 default SETING 설정 예시

```

function Recommend() {
  const sag = clamp(number(rec.sag.value, motor.defaultSag), 0, 40);
  const torque = Math.min(motor.peakTorque, positive(rec.torque.value, motor.defaultTorque));
  const currentLimit = positive(rec.dcl.value, motor.defaultDcl);
  const maxRPM = clamp(positive(rec.maxRPM.value, motor.maxRPM), 100, motor.maxRPM);
  const dclUsePct = clamp(number(rec.dclUse.value, SETTINGS.recommendationValidation.defaultDclUsePct), 50, 100);
  const targetVehicleSpeedKph = optionalPositive(rec.targetSpeed.value);
  const tireRadiusMm = optionalPositive(rec.tireRadius.value);
  const gearRatio = optionalPositive(rec.gearRatio.value);
  const vEff = vdc * (1 - sag / 100);
  const useRoadFit = rec.roadFit.checked;

  let kt = motor.ktTheory;
  let currentIntercept = 0;
  let currentSlope = Math.sqrt(3 / 2) * motor.ke;

  if (motorKey === "emrax208" && useRoadFit) {
    kt = motor.ktRoad;
    currentIntercept = SETTINGS.roadFit208.currentIntercept;
    currentSlope = SETTINGS.roadFit208.currentSlope;
  }

  if (motorKey === "emrax228" && useRoadFit) {
    currentSlope *= SETTINGS.roadFit208.currentTransferRatioFor228;
  }

  const iqPlateau = torque / kt;
  const electricalPowerLimit = vEff * currentLimit;

  // This follows the uploaded original:
  // Break Speed is limited by both electrical power and DC Bus Current.
  const breakByPower = 60 * electricalPowerLimit / (2 * Math.PI * torque);
  const breakByCurrent = (currentLimit - currentIntercept) * vEff / (currentSlope * iqPlateau);
  const breakRPM = Math.min(maxRPM, Math.max(0, breakByPower), Math.max(0, breakByCurrent));
  const mapPower = torque * omega(breakRPM);

  // Supplementary recommendation: calculate a conservative operating point at a user-selected
  // percentage of DCL. The original Break_Speed calculation above remains unchanged.
  const targetCurrent = currentLimit * dclUsePct / 100;
  const targetElectricalPower = vEff * targetCurrent;
  const safeBreakByPower = 60 * targetElectricalPower / (2 * Math.PI * torque);
  const safeBreakByCurrent = (targetCurrent - currentIntercept) * vEff / (currentSlope * iqPlateau);
  const safeBreakRPM = Math.min(maxRPM, Math.max(0, safeBreakByPower), Math.max(0, safeBreakByCurrent));

  // Vehicle kinematics validation. Empty optional inputs simply return null and do not affect
  // any existing parameter recommendation or graph result.
  const tireRadiusM = tireRadiusMm ? tireRadiusMm / 1000 : null;
  const vehicleMotorRPM = (targetVehicleSpeedKph && tireRadiusM && gearRatio)
    ? (targetVehicleSpeedKph / 3.6) * 60 * gearRatio / (2 * Math.PI * tireRadiusM)
    : null;
  const vehicleSpeedAtMaxRPM = (tireRadiusM && gearRatio)
    ? (maxRPM / gearRatio) * (2 * Math.PI * tireRadiusM) * 3.6 / 60
    : null;

  const currentAt = (rpm, iq) => Math.max(
    0,
    currentIntercept + currentSlope * rpm * iq / vEff
  );
}

```

그림 9. JS 파일에서 추천 파라미터 구현 연산

그림 10. JS 파일에서 커스텀 맵 구현

(자세한 코드 및 파일은 Github 링크 첨부)

4.2 추천 파라미터 설정 탭

추천 파라미터 설정 탭은 Motor Type, 배터리 전압, 전압 강하율, 최대 토크, DC Bus Current Limit, 권장 DCL 사용률, 목표 최고차속, 타이어 유효 반경, 총 감속비 및 예상 최대 RPM 을 입력받는다. 입력값이 변경되면 유효전압, Iq, Break_Speed, Break 시 기계 출력, 예상 DC Bus Current, DCL 사용률 기준 안전 운용 Break_Speed 를 다시 계산한다.

추천 탭의 전류 그래프는 기존 로직에 맞추어 $\min(\text{DCL}, \text{계산 전류})$ 로 제한 표시하므로, DCL 도달 여부를 직관적으로 확인하는 목적이다.

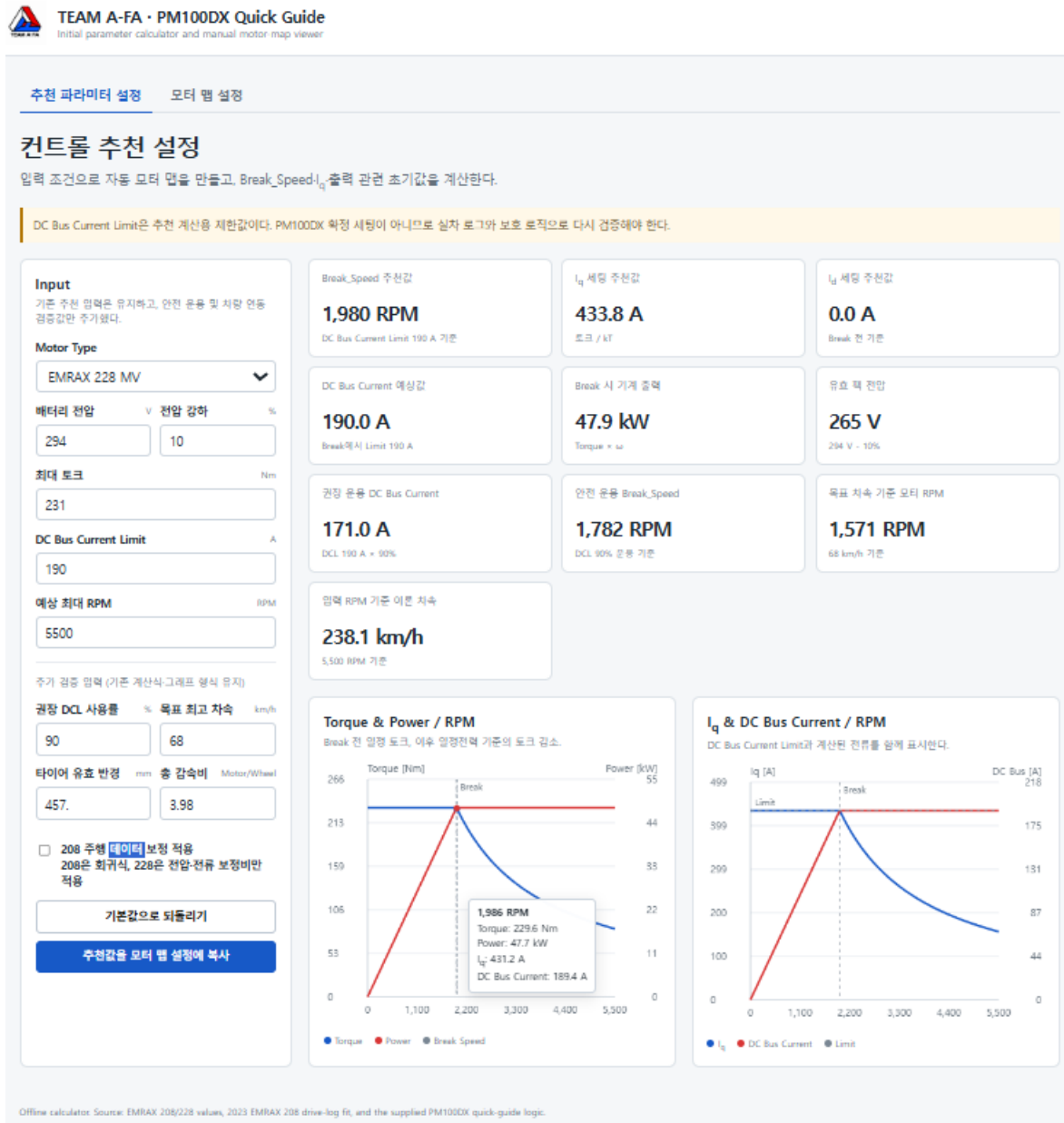


그림 8. 추천 파라미터 설정 화면

4.3 설정값 기반 추정 모터 맵 탭

수동 모터 맵 탭은 배터리 전압, 전압 강하율, Break_Speed, Iq Limit, Id Limit 및 Torque Limit 을 직접 입력하도록 구성하였다. 이 탭에서는 사용자가 입력한 모터 맵으로부터 DC Bus Current 를 계산하여 Peak DC Bus Current 와 DCL 검증 기준 대비 여유도를 확인한다.

수동 모터 맵의 Id Limit 은 Break_Speed 이후 점진적으로 음의 값으로 증가하는 표시용 프로파일로 구현하였다. 현재 버전에서는 검증된 Id-전압-전류 결합 모델이 없으므로, Id 는 그래프 툴팁과 CSV 에는 표시되지만 DC Bus Current 식에는 포함하지 않는다.



추천 파라미터 설정 **모터 맵 설정**

설정값 기반 추정 모터 맵

여기서는 직접 입력한 Break_Speed, I_q , I_d , Torque Limit만으로 모터 맵을 확인한다.

DC Bus Current는 이 맵의 입력 제한값이 아니다. 현재 설정값으로부터 계산되어 그래프와 결과값으로만 표시된다. 추가된 DCL 검증 기준은 비교용이며 계산값을 제한하지 않는다.



Offline calculator. Source: EMRAX 208/228 values, 2023 EMRAX 208 drive-log fit, and the supplied PM100DX quick-guide logic.

그림 9. 설정값 기반 추정 모터 맵 화면 예시

4.4 차량 제원 기반 차속 변환

차량 제원 입력은 DC Bus Current 회귀식의 직접적인 독립변수로 사용하지 않는다. 감속비와 타이어 반경은 모터 RPM 을 차량 속도로 변환하고, 입력한 Break_Speed 가 차량에서 어느 속도 영역에 해당하는지 판단할 수 있도록 기능을 추가하였다.

차량 질량, 구동계 효율, 타이어 마찰계수 등은 향후 구동력 및 가속도 예측 모델 확장 시 반영하여 연산을 진행한다면 더욱 정확한 값을 얻게 될 것이다.

$$v = (RPM_{motor} / ifinal) \times 2\pi r_{tire} \times 60 / 1000$$

4.5 한계 사항

본 도구의 핵심 한계는 다음과 같다.

- 228MV 주행 데이터의 부족으로 PM100DX 내부의 모든 제한 로직 모델링 불가
- I_d 를 전류·전압 계산에 직접 반영 불가

따라서 본 프로그램은 주행 데이터 보강 전까지 최종 세팅 값을 자동으로 확정하는 도구라고 볼 수 없고, 실차 적용 전 세팅 후보의 전류 여유도와 모터 맵 경향을 검토하는 초기 검증 도구로 사용해야 하며, 많은 주행 데이터가 확보되었을 때, 신뢰성 있는 세팅 값 확정기로 발전해 나갈 계획이다.

4.6 실차 검증 계획

실차 검증은 낮은 토크와 보수적인 DCL 에서 시작하여 단계적으로 진행한다. 각 단계에서 Vdc, sag, Iq, Id, Torque Feedback, Motor RPM, INV DC Bus Current, BMS DCL 데이터를 받아 선형회귀로 정리할 계획이다.

또한, 추가적으로 날씨 정보, 모터·인버터 온도도 가져와, 정리하여 PM100DX 의 온도 모델도 구현할 계획이다. 정리 계획 및 식 정리도는 다음과 같다.

그림 10. 실차 검증 절차 및 오차율 계산 검증 모델

▪ Manufacturing

5.1 개발 환경 및 산출물

분석 환경은 파이썬 기반 데이터 처리 및 선형회귀 환경, 실행 환경은 HTML/CSS/JavaScript 기반 브라우저 환경으로 구성하였다. 최종 산출물은

AFA_PM100DX_QuickGuide.html

AFA_PM100DX_QuickGuide.js

파일이다. 두 파일을 동일한 폴더에 배치한 뒤 HTML 파일을 열면 별도 설치 없이 실행된다.

5.2 구현 과정

첫째, 2023 년 208MV 주행 로그에서 전압·RPM·Iq·토크·DC Bus Current 를 정리하고 필터 조건을 적용하였다. 둘째, 파이썬을 활용하여 DC Bus Current 의 회귀 보정식을 구하고 PMSM 이론식 및 데이터시트 상수와 비교하였다. 셋째, 추천 로직과 수동 모터 맵 로직을 분리하여 JavaScript 계산 함수로 구현하였다. 넷째, 입력 오류를 줄이기 위해 모터별 토크·RPM 상한, DCL 사용률, 전압 강하율 및 설명 문구를 UI 에 반영하였다.

5.3 팀 내 사용 절차

팀원은 추천 파라미터 설정 탭에서 현재 팩 전압, 예상 전압 강하율, 목표 토크, DCL, 권장 DCL 사용률 및 예상 최대 RPM 을 입력한다. 이후 출력된 Break_Speed, Iq, 예상 DC Bus Current 및 안전 운용 Break_Speed 를 확인하고, 필요하면 추천값을 수동 모터 맵 탭으로 복사해 세팅 변화를 비교한다. 최종 값은 PM100DX 설정 화면에 직접 반영하기 전에 담당자가 실차 로그와 함께 재검토해야 한다.

5.4 인수인계 및 향후 개선

인수인계 시에는 각 상수의 출처, 회귀에 사용한 로그 파일, 필터 조건, 코드 버전 및 실차 검증 결과를 함께 보관한다. 향후 개선 항목으로는 모터 데이터베이스 JSON 분리, PM100DX CAN 로그 자동 불러오기, 모터·인버터 온도에 따른 보정, Id 를 포함한 약계자 전압 모델, 효율 맵, 차량 종방향 모델 및 복수 모터 맵 비교 기능이 있다. 특히 228MV 실차 로그를 확보하면 현재의 데이터시트 기반 초기 추정 구조를 228MV 전용 회귀식으로 갱신할 수 있도록 설계 및 제작을 완료했다.

■ Conclusion

본 설계에서는 PMSM 이론식과 2023 년 EMRAX 208MV 원본 CAN 로그 기반 회귀 보정식을 결합하여, PM100DX 초기 세팅 후보의 전류 여유도와 모터 맵 경향을 검토하는 HTML 기반 오프라인 도구를 구현하였다. 도구는 모터 종류, 배터리 조건, 토크, DCL 및 RPM 을 입력받아 Break_Speed 와 Iq 의 초기값을 계산하고, 일정 토크-일정 출력 모델을 기반으로 RPM 별 토크·출력·전류 변화를 시각화한다.

기존의 수작업 계산 방식과 비교할 때, 관련 파라미터를 한 화면에서 확인하고 추천값을 수동 모터 맵으로 복사해 즉시 비교할 수 있으므로 세팅 검토 시간과 입력 누락 가능성을 줄일 수 있다. 또한 DCL 사용률과 안전 운용 Break_Speed 를 별도로 제공하여, 초기 세팅 단계에서 과전류 위험을 사전에 검토할 수 있다.

다만 본 프로그램은 설계 보조 및 초기 검증 도구이며, 228MV 전용 회귀 데이터 부족, PM100DX 내부 제한 로직의 완전 구현, Id 표시 프로파일의 한계가 존재한다. 따라서 최종 적용 전에는 실제 BMS DCL, 배터리 전압 강하, 온도 및 PM100DX 로그를 활용한 단계별 실차 검증이 필수이다.

▪ Acknowledgments

기존 파라미터 세팅 및 계산 과정에 약 2 시간이 소요되었으나, 이번 이 프로그램을 사용하여 차량의 예상 출력 및 dcl 값을 설정 후 검토 및 검증까지 약 15 분 정도 소요되었다. 이번 모터 컨트롤 세팅 값 또한 프로그램을 통해 검증을 완료하여 파라미터 설정을 완료하였다.

▪ References

- [1] EMRAX, EMRAX 208 MV Datasheet.
- [2] EMRAX, EMRAX 228 MV Datasheet.
- [3] Cascadia Motion, PM100DX User Manual / Calibration Documentation.
- [4] 하라는코딩은안하고, 2023 EMRAX 208MV CAN Drive Log, 자작자동차포럼 (<https://fsk-endurance.luftaquila.io/>)
- [5] MathWorks, Permanent Magnet Synchronous Machine (PMSM) Documentation. (<https://kr.mathworks.com/help/sps/ref/pmsm.html>)
- [6] ChatGPT, 프론트 앤드 구현 prompt
()
- [7] 김종윤 외 1, 전기자동차용 PMSM 의 빠른 약자속 제어 방법, 한국자동차공학회, 2017 추계학술대회, 2017